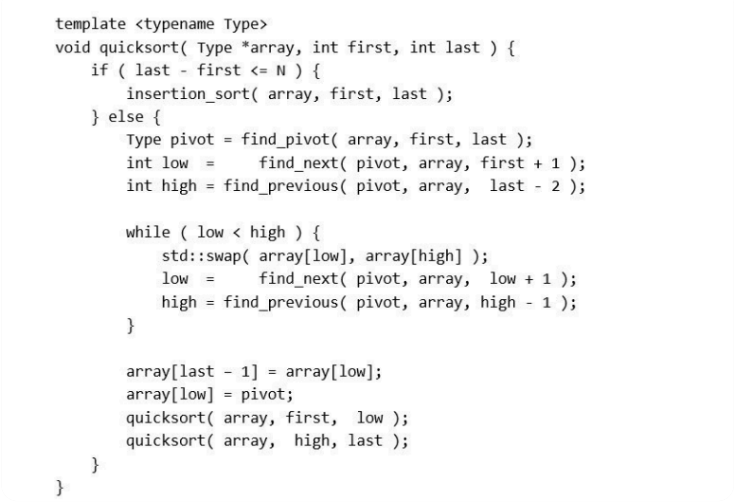
# ChatGPT - DSA swe



```
template <typename Type>
void quicksort( Type *array, int first, int last ) {
    if ( last - first <= N ) {
        insertion_sort( array, first, last );
    } else {
        Type pivot = find_pivot( array, first, last );
        int low  =     find_next( pivot, array, first + 1 );
        int high = find_previous( pivot, array,  last - 2 );

        while ( low < high ) {
            std::swap( array[low], array[high] );
            low  =     find_next( pivot, array,  low + 1 );
            high = find_previous( pivot, array, high - 1 );
        }

        array[last - 1] = array[low];
        array[low] = pivot;
        quicksort( array, first,  low );
        quicksort( array,  high, last );
    }
}
```

wrtite a code for integer quicksort in this manner and explain every part of the coder

Sure! Based on the template code pattern from your image, here's a complete **quicksort implementation for integers**, using a similar structure (with `find_pivot`, `find_next`, `find_previous`, and `insertion_sort` as helpers), followed by a **step-by-step explanation**:

---

## ✅ C++ Code: Integer Quicksort with Helper Functions

cpp

```cpp
#include <iostream>
#include <algorithm>

const int N = 10;  // Threshold for using insertion sort

// Simple insertion sort for small arrays
void insertion_sort(int array[], int first, int last) {
    for (int i = first + 1; i < last; ++i) {
        int key = array[i];
        int j = i - 1;
        while (j >= first && array[j] > key) {
            array[j + 1] = array[j];
            --j;
        }
        array[j + 1] = key;
```

```
    }
}

// Find pivot as median of first, middle, last
int find_pivot(int array[], int first, int last) {
    int mid = (first + last - 1) / 2;
    int a = array[first];
    int b = array[mid];
    int c = array[last - 1];
    if ((a < b && b < c) || (c < b && b < a)) return b;
    else if ((b < a && a < c) || (c < a && a < b)) return a;
    else return c;
}

// Find next index from left where value >= pivot
int find_next(int pivot, int array[], int start) {
    while (array[start] < pivot) ++start;
    return start;
}

// Find previous index from right where value <= pivot
int find_previous(int pivot, int array[], int start) {
    while (array[start] > pivot) --start;
    return start;
}

// Quicksort function
void quicksort(int array[], int first, int last) {
    if (last - first <= N) {
        insertion_sort(array, first, last);
    } else {
        int pivot = find_pivot(array, first, last);
        int low = find_next(pivot, array, first + 1);
        int high = find_previous(pivot, array, last - 2);

        while (low < high) {
            std::swap(array[low], array[high]);
            low = find_next(pivot, array, low + 1);
            high = find_previous(pivot, array, high - 1);
        }

        array[last - 1] = array[low];
```

```
        array[low] = pivot;

        quicksort(array, first, low);
        quicksort(array, high, last);
    }
}

// Main to test
int main() {
    int data[] = {34, 7, 23, 32, 5, 62, 19, 4, 8, 56, 0, 17};
    int size = sizeof(data) / sizeof(data[0]);

    quicksort(data, 0, size);

    std::cout << "Sorted array: ";
    for (int i = 0; i < size; ++i)
        std::cout << data[i] << " ";
    return 0;
}
```

## 🧠 Step-by-step Explanation

🔸 `insertion_sort(...)`

- For small subarrays (length $\leq$ N), we use insertion sort.
- This is a simple O($n^2$) sort ideal for small inputs.

🔸 `find_pivot(...)`

- Returns the **median of first, middle, and last elements** to avoid worst-case pivot choice (like already sorted input).
- This improves performance by better partitioning.

🔸 `find_next(...)`

- Starts from `start` and goes right until it finds an element **not less than the pivot**.
- Helps in correctly placing values ≥ pivot to the right.

---

🔸 `find_previous(...)`

- Starts from `start` and goes left until it finds an element **not greater than the pivot**.
- Helps in correctly placing values ≤ pivot to the left.

---

🔸 `quicksort(...)`

- If the array length is small, switch to insertion sort for efficiency.
- Else, pick pivot → partition using two pointers ( `low` , `high` ) and swap values across them.
- Finalize the pivot position, then **recursively sort** left and right halves.

---

🔸 `main(...)`

- Declares an unsorted array.
- Calls `quicksort(...)` .
- Prints the sorted array.

---

Would you like a diagram or trace output of how the sort works on a sample array (like with 62 you asked earlier)?

now write an easy to understand pseudo code with explanation for this

Sure! Here's an **easy-to-understand pseudocode** version of the integer quicksort implementation, **following the same logic as your C++ code**, along with **step-by-step explanations** for each part.

## ✅ Pseudocode: Quicksort with Helper Functions

sql

```
FUNCTION quicksort(array, first, last):
    IF last - first <= N:
        CALL insertion_sort(array, first, last)
    ELSE:
        pivot ← find_pivot(array, first, last)
        low ← find_next(pivot, array, first + 1)
        high ← find_previous(pivot, array, last - 2)

        WHILE low < high:
            SWAP array[low] and array[high]
            low ← find_next(pivot, array, low + 1)
            high ← find_previous(pivot, array, high - 1)

        array[last - 1] ← array[low]
        array[low] ← pivot

        CALL quicksort(array, first, low)
        CALL quicksort(array, high, last)
```

## 🧩 Helper Functions

sql

```
FUNCTION insertion_sort(array, first, last):
    FOR i FROM first + 1 TO last - 1:
        key ← array[i]
        j ← i - 1
        WHILE j >= first AND array[j] > key:
            array[j + 1] ← array[j]
```

```
            j ← j - 1
        array[j + 1] ← key
```

```sql

FUNCTION find_pivot(array, first, last):
    mid ← (first + last - 1) / 2
    RETURN median of array[first], array[mid], array[last - 1]
```

```sql

FUNCTION find_next(pivot, array, start):
    WHILE array[start] < pivot:
        start ← start + 1
    RETURN start
```

```sql

FUNCTION find_previous(pivot, array, start):
    WHILE array[start] > pivot:
        start ← start - 1
    RETURN start
```

---

## 📘 Explanation of Each Part

---

✅ `quicksort(...)`

- Main recursive sorting function.
- If the subarray is small (length ≤ N), use `insertion_sort`.
- Otherwise:
  - Choose a **pivot** value.
  - Initialize two pointers: `low` moves right, `high` moves left.
  - Swap elements to move values smaller than pivot to the left, larger to the right.

- Finalize pivot position.
- Recursively apply quicksort on the left and right partitions.

---

### 🔧 `insertion_sort(...)`

- A simple sorting algorithm used when the subarray is very small.
- It is efficient for small data due to low overhead.

---

### 🎯 `find_pivot(...)`

- Picks a good pivot to reduce chances of worst-case performance.
- Uses **median-of-three**: first, middle, and last elements.

---

### 🔍 `find_next(...)`

- Finds the next element **from left** that is not less than the pivot.

---

### 🔍 `find_previous(...)`

- Finds the next element **from right** that is not greater than the pivot.

---

Let me know if you'd like a **step-by-step dry run** with an example array (e.g., `[34, 7, 23, 62]`) to see how the algorithm works visually.